

ARCHITECTURE DESIGN

CPF 아키텍처 설계서

Module Ownership·Topology·Runtime·Security·Recovery·Observability 구조를 설계합니다.

Core Platform Framework · Source-aligned user documentation

주 독자 아키텍트 · Tech Lead · 플랫폼 개발자

01 Module/Owner/Dependency 방향 확인	02 Local/Remote·MSA·다중 인스턴스 구조 확인	03 Transaction/UNKNOWN/Recovery/C ontrol Plane 흐름 확인	
SOURCE	SOURCE REWORK	RUNTIME	RELEASE
4870b2073387 · 07_06	34/34 반영 · 신규 31 Action	0/13 live 실행	RELEASE_BLOCKED

07_06 Final QA 반영

● EDU-ADM 9 PRODUCT / 4 EXTENSION / 4 MERGE	● Control Plane action permission 보강
● transaction lineage one-shot Source 보강	● FileLog durable recovery Owner 추가

PDF는 공식 열람본, DOCX는 동일 내용의 편집 원본입니다. 현재 Source 보완과 Runtime/QA 상태를 구분해 읽으십시오.

빠른 찾기

긴 문서를 처음부터 읽기보다 아래 흐름과 장 목록에서 필요한 지점을 먼저 찾으십시오. PDF Bookmarks와 검색 (Ctrl+F)도 함께 사용할 수 있습니다.

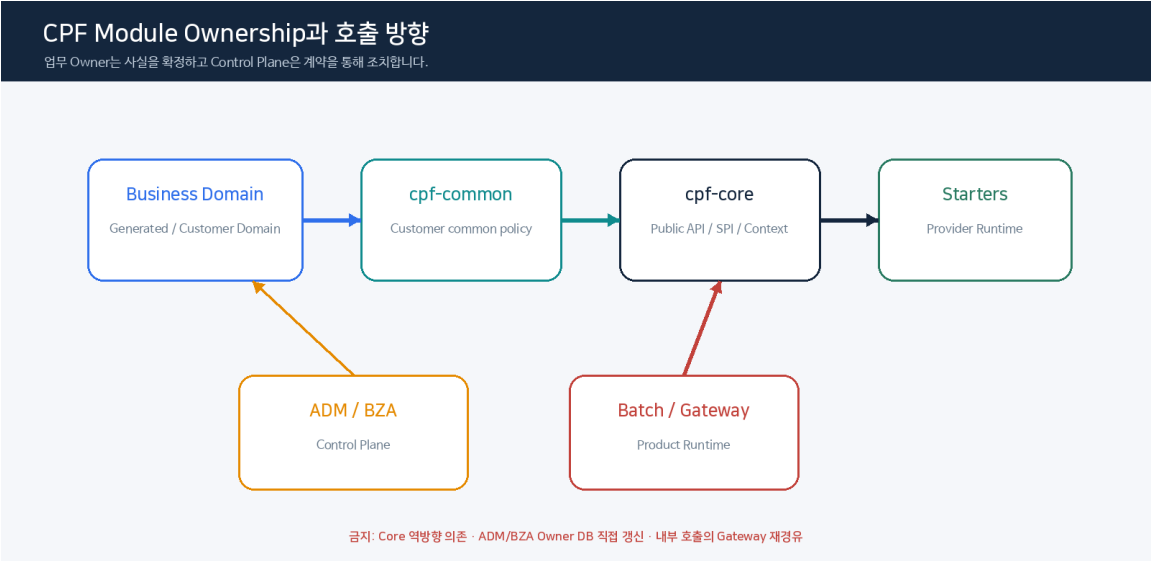


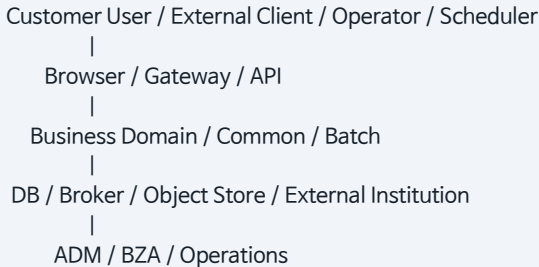
그림 1. 이 문서의 핵심 업무 흐름

NAV	제목	NAV	제목
01	System Context	09	ADM Control Plane
02	Module Ownership 상세	10	BZA Control Plane
03	Deployment Topology	11	Security Architecture
04	Local·Remote 동등성	12	DB Vendor Architecture
05	Online Transaction Flow	13	Observability Architecture
06	Async Architecture	14	Deployment Architecture
07	Batch Architecture	15	EDU Reference Architecture
08	Gateway Data/Control Plane		

1. 설계 목표

CPF 는 업무 개발·운영·감사·복구·배포를 같은 계약으로 연결한다. 품질 속성은 변경 용이성, 일관성, 복구 가능성, 보안, 운영성, 이식성, 추적성이다.

2. System Context



Trust Boundary: Browser-BFF, External-Gateway, Service-Service, Runtime-Infra, Operator-Control Plane.

3. Module View

cpf-core, cpf-common, cpf-starters, cpf-batch, cpf-admin, cpf-biz-admin, cpf-gateway, cpf-tools, cpf-member, cpf-reference 의 책임·Consumer·금지 의존은 00 과 동일하게 유지한다.

4. Public API·SPI·Internal

Public API 는 호환성 대상, SPI 는 Provider 확장 계약, Internal 은 외부 Compile 을 금지한다. API/SPI 는 lifecycle, failure, threading, idempotency, versioning 을 문서화한다.

5. Profile·Capability Architecture

6 개 Public Profile 과 Capability Group 을 조립하고 선택 결과를 Manifest/Starter Lock 으로 고정한다. 선택하지 않은 Provider 의 JAR/Bean/SQL/Config/Secret 요구가 없어야 한다.

6. Online Transaction Flow

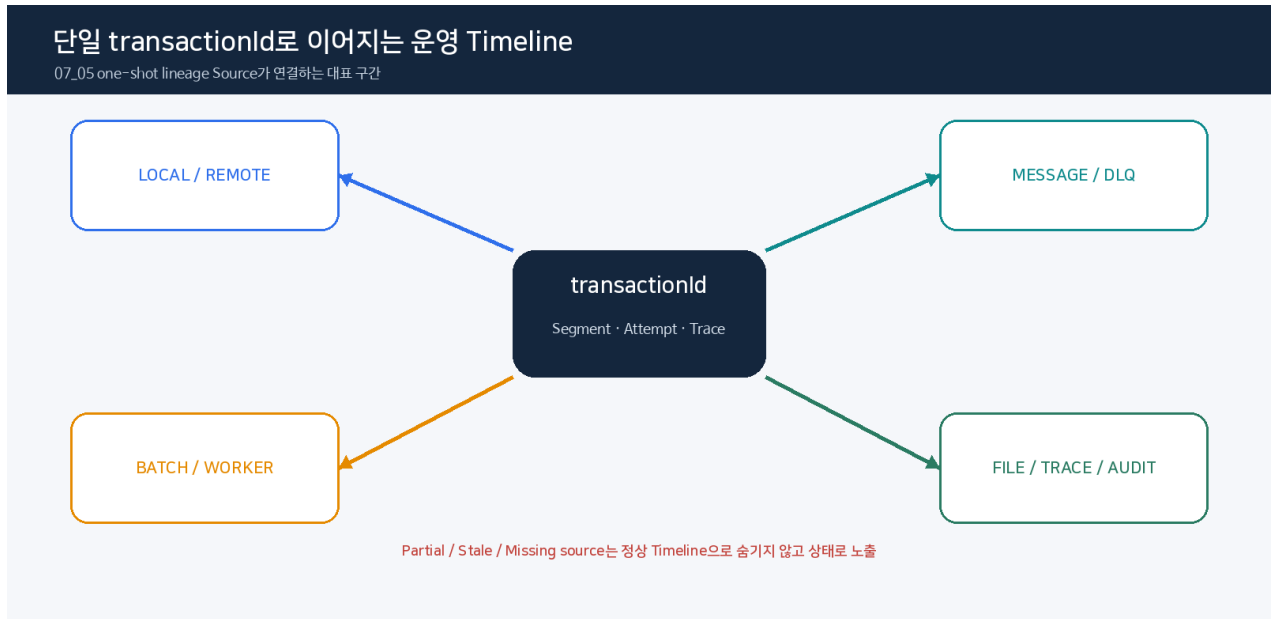


그림. Cross-runtime transaction lineage

Authentication→Context→Validation→Permission/Scope→Idempotency/Hash→Domain/
Version→Current+History+Audit+Outbox→Commit→Result→후속 Attempt.

7. Async Architecture

Outbox Producer, Publisher Lease/Attempt, Broker, Consumer Inbox, 업무 Commit, ACK, DLQ/Replay. Multi-instance 는 Lease/Fencing 으로 stale writer 를 차단한다.

8. Batch Architecture

Spring Batch Metadata + CPF Scheduler/Runner/Worker/Agent/Center-Cut/Recovery. 기술 완료와 업무 합계를 분리한다.

9. ADM Control Plane

Route Registry + Generated Client + Owner Port. 위험 조치는 Reason/Version/Approval/Idempotency. UNKNOWN/PARTIAL/DRIFT 를 운영 언어로 사용한다.

10. BZA Control Plane

조직/사용자/Role/Permission/Scope/Masking/Approval 을 기준일과 Version 으로 제공하며 업무 상태는 Domain 이 소유한다.

11. Gateway Data/Control Plane

Data Plane: listener/predicate/rewrite/security/target/attempt. Control Plane: draft/validate/approve/publish/ACK/NACK/LKG/rollback/reconcile.

12. Data Architecture

Current, History, Idempotency, Attempt, Outbox/Inbox/DLQ, Approval Snapshot, Audit, Reconciliation Decision 을 Owner 별로 소유한다.

13. Error Taxonomy

Validation, Authentication, Authorization, Conflict, Dependency, Unknown, Partial, Drift 를 구분하고 각 오류에 Retry/Reconcile/Rollback 의미를 부여한다.

14. Security Architecture

User/Service Identity, Session/CSRF/MFA, Gateway HMAC/Nonce, Menu/Button/API/Scope/Masking, Secret Rotation, Approval/Break-glass, Immutable Audit.

15. Deployment Architecture

Embedded JAR, External WAS, Modular Monolith, Microservice, Static Frontend, Gateway Runtime, Agent/Runner/Worker, Multi-instance/Multi-zone, Rolling/Canary/Blue-Green.

16. Failure Architecture

실패	보호	정상화
Process Kill	Transaction/Checkpoint/Lease	Restart/Reclaim/Reconcile
Network Partition	Timeout/Circuit/Fencing	Target/Owner Reconcile
Response Loss	Attempt/Operation	Status Query/Compensation
Partial Apply	Target ACK/Observed	Reapply/LKG
DB Migration Partial	Backup/Version/Verify	Forward Recovery/Restore
Secret Rotation Partial	Grace Version/Consumer State	실패 Consumer 재적용

17. Observability Architecture

Transaction/Trace/Operation/Attempt/Target/Actor 를 연결하며 PII/Secret 원문을 Log/Metric/Trace/Evidence 에 남기지 않는다.

18. Architecture Decision Checklist

Owner, Consumer, topology, failure semantics, consistency, security, recovery, observability, deployment, compatibility, verification 을 변경마다 검토한다.

19. Saga Architecture

Saga State Store 는 각 Step 의 Owner 상태를 대체하지 않고 correlation/timeout/compensation/manual decision 을 연결한다. Step 결과 불명은 전체 Saga 실패로 단순화하지 않는다.

20. Resource Architecture

모든 실행 경로는 bounded memory/disk/thread/connection/queue/time budget 을 가진다.
File/Archive/Message/Support Bundle 에서 전체 payload 적재를 피하고 streaming/backpressure 를 사용한다.

21. Configuration Architecture

Typed config, metadata, profile, env, secret reference, dynamic desired/observed state 를 분리한다. 변경은 version/checksum/approval/target apply/reconcile/rollback lifecycle 을 가진다.

22. Data Lineage·Retention Architecture

Dataset/Field lineage 를 Source→Transform→Store→External 로 연결하고 data quality/reconcile/retention/purge/archive/legal hold 를 Owner 별로 정의한다.

23. Supply-chain Architecture

Source→Build→Artifact→Manifest→Signature→Registry/Offline Bundle→Deploy Target 의 Hash Chain 을 유지한다.
SBOM/License/Vulnerability/Provenance 는 최종 Artifact 기준으로 생성한다.

24. Compatibility Architecture

Local/Remote, DB Vendor, Mixed Application Version, Schema Version, Message Schema, Config Default, Browser Client Version 을 독립 축으로 관리한다.

25. Recovery Decision Model

```

실패 감지
-> 부작용 전인가? ---- yes -> FAILED / bounded retry
-> 부작용 후인가? ---- yes -> 결과 Evidence 충분? -- yes -> 확정 상태
    |
    no
    v
  UNKNOWN_RESULT
    |
status query / reconcile
    |
succeed / fail / compensate / manual
  
```

26. 현재 Product Surface Architecture Matrix

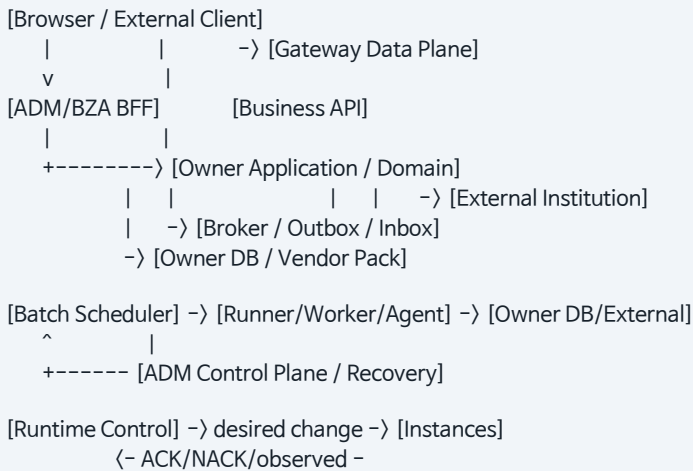
Surface	정본	Architecture 의미
Starter	cpf-tools/generator/contracts/cpf-starter-catalog.json	39 active + 1 retained inactive; 6 profile; 7 capability group
EDU	cpf-reference/src/main/resources/edu/manual-135-catalog.json	135 정상·장애·복구 Reference 계약
Generator	cpf-tools/generator/create-domain.ps1	Profile/Capability/Provider/DB/ownership deterministic generation

Surface	정보	Architecture 의미
Tool	cpf-tools/README.md, cpf-tools/scripts/*	Build/DB/Generator/verification/Evidence 지원
Batch	cpf-batch/* 9 subproject	Spring Batch Primary Engine + Control/Scheduler/Worker/Center-Cut/Agent
ADM/BZA	Backend + Vue frontend	Owner Query/Command Control Plane, 권한/승인/감사
Gateway	cpf-gateway	외부 Trust Boundary + Route/Attempt/LKG

Architecture 변경은 위 정보 중 하나만 수정해 끝내지 않는다. Source/Config/SQL/API/Test/Guide/Evidence 의 영향을 함께 추적한다.

27. Architecture Context 상세

CPF 는 단일 배포형태를 전제로 하지 않는다. 같은 Business Contract 를 Modular Monolith, 분리 Service, External WAS, Batch Worker, Gateway Runtime 에서 재사용하되 각 topology 가 추가하는 장애·보안 경계를 명시한다.



각 화살표는 단순 네트워크 연결이 아니라 Identity, Timeout, Retry, Correlation, Failure/Recovery 계약을 가진다.

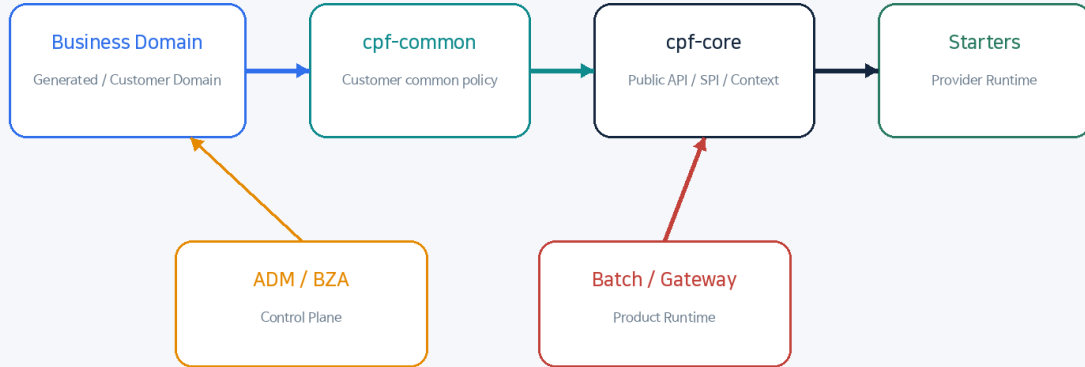
28. Actor·Trust Boundary

Actor/Boundary	인증·신뢰	주요 위험	Architecture 보호
Browser Operator	Server Session	CSRF, stale permission, PII 노출	Trusted Origin, CSRF, HttpOnly Session Handle, masking
External Client	Gateway AuthN/AuthZ	SSRF, replay, spoofing, overload	Route Default Deny, HMAC/JWT, nonce, TLS, rate limit
Service-to-Service	Service Identity	caller spoofing, key rotation	Service ID/key version, audience, secret provider
Batch Worker/Agent	Runtime Identity	stale worker, command replay	Lease/Fencing, idempotency, approval snapshot
DB/Broker/Storage	Infra credential	privilege leakage, partial failure	Secret reference, least privilege, attempt/reconcile
Operator Control Plane	elevated privilege	dangerous mutation	Menu/API/Button permission, Reason, Approval, Audit

29. Module Ownership 상세

CPF Module Ownership과 호출 방향

업무 Owner는 사실을 확정하고 Control Plane은 계약을 통해 조치합니다.



금지: Core 역방향 의존 · ADM/BZA Owner DB 직접 갱신 · 내부 호출의 Gateway 재경유

그림. Module Ownership 과 허용 의존 방향

Module/Area	소유하는 상태/책임	대표 Consumer	금지 의존
cpf-core	Public API/SPI, cross-cutting runtime contracts	모든 CPF module	Provider SDK 구체 타입을 Public contract 에 노출
cpf-common	공통 업무/Platform 구현	business modules	개별 업무 Domain 상태 소유
cpf-starters	Capability Provider binding/AutoConfiguration	generated/customer domain	선택되지 않은 Provider 강제 활성화
cpf-batch	Job/Schedule/Execution/Worker/Center-Cut/Agent	batch app, ADM	업무 Domain Table 직접 상태 소유
cpf-admin	Platform 운영 Query/Command facade + ADM UI	operator	Owner 상태를 ADM DB 복제본으로 대체
cpf-biz-admin	BZA 조직/사용자/권한/결재	BZA UI/domain consumers	업무 Domain 상태 소유
cpf-gateway	external route/control/data plane	external client/services	Service Registry 존재만으로 공개
cpf-tools	generator/DB/build/verification/document support	developer/operator/CI	Working Tree 무조건 덮어쓰기
cpf-reference	135 EDU executable/reference contract	교육/검증	제품 Public API 의 제 2 정본 역할
business domains	실제 업무 Current/History/invariant	API/Batch/ADM	다른 Domain 내부 DB 직접 DML

30. Dependency Direction

의존은 Consumer -> Public API/SPI -> Owner implementation 방향을 유지한다. Admin/Frontend 가 Owner Repository 를 직접 호출하지 않고 Owner Port/API 를 사용한다. Remote topology 로 분리해도 DTO/Validation/Error/Idempotency 의미가 달라지지 않도록 한다.

Frontend -> Generated Client -> ADM/BZA/Gateway Backend -> Owner Port
 Business App -> cpf-core API/SPI <- Starter Provider
 Batch Workbench -> Batch Owner API -> Spring Batch/CPF Ledger

Internal package 는 compile convenience 때문에 Public Surface 로 승격하지 않는다.

31. Public Profile Architecture

현재 공개 Profile 은 6 개이며 Starter Catalog 정본을 따른다.

Profile 계열	Architecture 목적	선택 시 확인
minimal-domain	최소 Domain boot	foundation/base, DB/필수 runtime 만
web-api	HTTP API	web/OpenAPI/observability 조합
secure-api	인증된 API	resource server/service identity/secret
browser-bff	privileged browser	session/CSRF/operator security
event-service	broker/event consumer-producer	messaging provider + reliability
batch-service	Spring Batch runtime	batch engine/scheduler/worker/operations

Profile 이름은 고객 기능을 숨기는 마법 설정이 아니라 reproducible starter composition 입력이다. 실제 Provider 선택과 generated manifest 를 함께 확인한다.

32. Capability Group Architecture

현재 7 개 Capability Group 은 data, messaging, integration, file, notification, security, platform-operations 다. Group 은 Provider Slot 의 논리 분류이며 실제 Runtime Property Prefix 의 정본이 아니다.

Provider 를 교체할 때 Consumer 는 Public API/SPI 에 남고, Provider-specific AutoConfiguration/Config/Health/Test 가 교체된다.

33. Starter Runtime Binding Matrix

Starter	Catalog Namespace	Runtime Surface	분류	의미
cpf-starter-data-cache-caffeine	cpf.data.cache.caffeine	cpf.data.cache.caffeine.provider + cpf.cache.caffeine.*	DIRECT_LEAF_DIFFERENT_PREFIX	Provider condition 과 runtime property prefix 분리
cpf-starter-data-cache-valkey	cpf.data.cache.valkey	cpf.data.cache.valkey.*	DIRECT_LEAF	Valkey runtime properties
cpf-starter-data-persistence-jdbc	cpf.data.persistence.jdbc	cpf.data.persistence.jdbc.* + cpf.db.* + spring.datasource.cpf.*	DIRECT_LEAF_PLUS_CORE_DB	JDBC/DB vendor/resource /routing
cpf-starter-data-persistence-mybatis	cpf.data.persistence.mybatis	cpf.db.* + cpf.datasource; MyBatis direct CPF leaf 없음	CONSUMER_OF_CORE_DB	CpfMyBatisConfig uses CpfSqlResourceResolver
cpf-starter-file-archive	cpf.file.archive	cpf.file.archive.*	DIRECT_LEAF	archive bounds
cpf-starter-file-attachment	cpf.file.attachment	cpf.framework.attachment.*	DIRECT_LEAF_DIFFERENT_PREFIX	legacy/current runtime prefix differs from catalog namespace
cpf-starter-file-sftp	cpf.file.sftp	cpf.file.sftp.*	DIRECT_LEAF	SFTP + secret ref + ledger
cpf-starter-file-tabular-poi	cpf.file.tabular.poi	직접 CPF leaf 없음	NO_DIRECT_LEAF	CSV/XLSX adapter bean; limits belong call contract
cpf-starter-foundation-base	cpf.foundation.base	cpf.starter.base.*	DIRECT_LEAF_DIFFERENT_PREFIX	base strict/profile id/version
cpf-starter-integration-fixedlength	cpf.integration.fixedlength	직접 main runtime leaf 없음	NO_DIRECT_LEAF	Codec registry/layout contract

Starter	Catalog Namespace	Runtime Surface	분류	의미
cpf-starter-integration-fixedlength-core	cpf.integration.fixedlength.core	직접 leaf 없음	PURE_INTERNAL_CODE C	상위 fixedlength 내부 engine
cpf-starter-integration-http-client	cpf.integration.http	cpf.service-call.* + cpf.http-client.* + cpf.services.(serviceId).*	DIRECT_LEAF_DIFFERENT_PREFIX	service call, timeout, endpoint SSRF/pinning
cpf-starter-integration-iso8583	cpf.integration.iso8583	직접 운영 leaf 를 문서에서 생성하지 않음	PROTOCOL_CONTRACT	message/layout 계약 중심; 추정 key 금지
cpf-starter-integration-resilience	cpf.integration.resilience	cpf.integration.resilience.* + cpf.reconciliation.worker.id	DIRECT_LEAF	lock/state/guard/reconcile
cpf-starter-integration-tcp	cpf.integration.tcp	cpf.integration.tcp.*	DIRECT_LEAF	connection/framing/TLS/backpressure/unknown journal
cpf-starter-integration-webhook	cpf.integration.webhook	직접 Spring runtime leaf 없음	PURE_API_LIBRARY	current build depends on cpf-core only; customer adapter owns endpoint/secret
cpf-starter-messaging-ibm-mq	cpf.messaging.ibm-mq	cpf.messaging.ibm-mq.*	DIRECT_LEAF	CCDT 또는 channel+connection, customer ConnectionFactory
cpf-starter-messaging-jms	cpf.messaging.jms	cpf.messaging.jms.*	DIRECT_LEAF	JMS destination/session/ack
cpf-starter-messaging-kafka	cpf.messaging.kafka	cpf.messaging.kafka.*	DIRECT_LEAF	binding/ack/payload
cpf-starter-messaging-rabbitmq	cpf.messaging.rabbitmq	cpf.messaging.rabbitmq.*	DIRECT_LEAF	exchange/queue/manual ack/confirm
cpf-starter-messaging-reliability-jdbc	cpf.messaging.reliability.jdbc	cpf.messaging.reliability.*	DIRECT_LEAF_DIFFERENT_PREFIX	outbox/inbox/dlq/lease/replay/reconcile
cpf-starter-notification-dispatch	cpf.notification.dispatch	cpf.notification.dispatch.*	DIRECT_LEAF	worker/lease/retry/reconcile
cpf-starter-notification-email	cpf.notification.email	cpf.notification.email.enabled/from + Spring Mail provider config	DIRECT_LEAF_PLUS_PROVIDER	JavaMailSender external provider
cpf-starter-notification-sms-spi	cpf.notification.sms	Starter 자체 direct leaf 없음	EXTENSION_SPI	customer SMS provider adapter owns endpoint/credential
cpf-starter-platform-operations-channel-registry-jdbc	cpf.platform-operations.channel-registry.jdbc	cpf.channel-policy.startup-load-enabled + DB runtime	DIRECT_LEAF_DIFFERENT_PREFIX	JDBC registry
cpf-starter-platform-operations-feature-flag-openfeature	cpf.platform-operations.feature-flag	cpf.platform-operations.feature-flag.enabled	DIRECT_LEAF	JDBC state/audit + hot apply
cpf-starter-platform-operations-observability	cpf.platform-operations.observability	cpf.log-policy.cache.* + cpf.log-policy.default.* + DB managed policy	DIRECT_LEAF_DIFFERENT_PREFIX	policy cache/capture/masking
cpf-starter-platform-operations-otlp	cpf.platform-operations.otlp	cpf.platform-operations.otlp.*	DIRECT_LEAF	endpoint/timeout/sample
cpf-starter-platform-operations-runtime-control-client	cpf.platform-operations.runtime-control	cpf.runtime.control.* + cpf.runtime.*	DIRECT_LEAF_DIFFERENT_PREFIX	local/remote control, registration, guard, reconcile
cpf-starter-profile-batch-service	cpf.profile.batch-service	Profile composition only	PROFILE_COMPOSITION	leaf belongs resolved starters
cpf-starter-profile-browser-bff	cpf.profile.browser-bff	Profile composition only	PROFILE_COMPOSITION	leaf belongs resolved starters

Starter	Catalog Namespace	Runtime Surface	분류	의미
cpf-starter-profile-event-service	cpf.profile.event-service	Profile composition only	PROFILE_COMPOSITION	leaf belongs resolved starters
cpf-starter-profile-minimal-domain	cpf.profile.minimal-domain	Profile composition only	PROFILE_COMPOSITION	leaf belongs resolved starters
cpf-starter-profile-secure-api	cpf.profile.secure-api	Profile composition only	PROFILE_COMPOSITION	leaf belongs resolved starters
cpf-starter-profile-web-api	cpf.profile.web-api	Profile composition only	PROFILE_COMPOSITION	leaf belongs resolved starters; openapi-webmvc internalized
cpf-starter-security-resource-server	cpf.security.resource-server	cpf.security.resource-server.*	DIRECT_LEAF	issuer/JWK XOR + audience
cpf-starter-security-secret	cpf.security.secret	직접 leaf 없음; approved CpfSecretProvider required	PROVIDER_SPI_REQUIRE D	no provider => startup failure
cpf-starter-security-service-identity	cpf.security.service-identity	cpf.security.service-identity.*	DIRECT_LEAF	key rotation/service token
cpf-starter-security-session-jdbc	cpf.security.session.jdbc	cpf.security.session.*	DIRECT_LEAF_DIFFERENT_PREFIX	session/vault/csrf/CSP/fail-closed

이 Matrix 는 논리 Catalog 와 Runtime 설정이 다른 사례 (Caffeine, Session, HTTP 등) 를 Architecture 수준에서 명시해 Config drift 를 방지한다.

34. Online Data Plane

Ingress
 -> Identity/Context
 -> Validation
 -> Permission/Scope/Masking policy
 -> Idempotency/Request Hash
 -> Owner Application
 -> Domain invariant + Expected Version
 -> Persistence (Current/History/Audit/Outbox)
 -> Commit
 -> Response

응답이 Client 에 도달하지 않았다는 사실과 Owner Commit 여부는 별개다. Commit 이후 응답 유실은 UNKNOWN_RESULT 후보이며 Operation/Idempotency Ledger 로 대사한다.

35. Local·Remote 동등성

Local same-JVM 호출은 network timeout/serialization 이 없지만 Remote 가 됐을 때도 업무 의미를 보존해야 한다.

의미	Local	Remote 추가 요소
Validation	같은 request rule	serialization/schema validation
Identity	context 전달	signed/service identity header
Timeout	call budget	connect/read/overall timeout
Failure	exception	HTTP/protocol error mapping
Idempotency	same key/hash	header/body contract 전파
Audit	same owner audit	caller/attempt metadata 추가

Local 전용 Internal Class 를 Remote Public API 처럼 문서화하지 않는다.

36. Transaction·Consistency Architecture

한 Owner DB 의 Current/History/Audit/Outbox 는 필요 시 같은 Local Transaction 으로 묶을 수 있다. Broker, External HTTP, File Storage 등 다른 Resource 의 Side Effect 를 DB rollback 과 동일하다고 보지 않는다.

Strong consistency 가 필요한 상태 전이는 Owner Version/CAS 로 보호하고, 분산 결과는 Attempt/Reconcile/Compensation 으로 달는다.

37. Messaging Architecture

```

Owner TX: Business + Outbox
|
Publisher Lease/Fencing
|
Broker Attempt --- timeout ---> UNKNOWN publish status
|           |
ACK         broker/status reconcile
|
Consumer -> Inbox duplicate guard -> Business TX -> ACK
|
failure -> Retry -> DLQ -> approved Replay
  
```

Kafka/RabbitMQ/JMS/IBM MQ Provider 차이는 Adapter 에서 흡수하고 Reliability Ledger 는 Provider 독립 의미를 유지한다.

38. External Integration Architecture

HTTP/TCP/SFTP/Webhook/ISO8583 은 Target/Attempt/Timeout/Tracking/Result 를 공통 축으로 본다. External Institution 의 응답 유실 가능성이 있으면 status query 또는 reconciliation port 가 Architecture 의 일부다.

HTTP 는 Service Registry/Endpoint 또는 cpf.services.* 같은 승인 설정을 사용하며 DNS/Private/Public/CIDR/Port/Pin 정책으로 SSRF 경계를 둔다. SFTP 는 Secret Reference 를 사용한다.

39. File Architecture

Attachment/File Job/Archive/Tabular 는 Streaming 과 Artifact lifecycle 을 중심으로 설계한다.

```

Upload/Acquire
-> size/schema/checksum validation
-> temp/quarantine
-> scan/parse/row validation
-> dry-run or approval
-> apply/dispatch
-> result ledger
-> artifact/retention
-> rollback/reconcile if supported
  
```

대용량 파일을 Browser/Heap 전체 메모리에 적재하지 않는다.

40. Batch Component Architecture

Component	역할	장애/복구 핵심
Spring Batch Engine	Job/Step/Tasklet/Chunk/Metadata	Checkpoint/Restart

Component	역할	장애/복구 핵심
Job Definition	Versioned 실행 계약	DRAFT→VALIDATED→APPROVAL→PUBLISHED/RETIRED
Scheduler	due/misfire/calendar 판단	leader/lease/duplicate trigger
Runner	Job 시작·파라미터 고정	operation/idempotency
Worker	Partition/Task 실행	heartbeat/lease/fencing/reassignment
Agent	Host process/artifact control	approval, snapshot hash, rollback
Center-Cut	대량 target/result 처리	target-level retry/reconcile
Recovery	Ghost/Unknown/Lock	manual decision/compensation
ADM Workbench	operator view/control	reason/approval/version/audit

41. Batch Failure Domains

Worker Process kill, Scheduler failover, Broker dispatch loss, DB lock, external writer response loss 를 서로 다른 Failure Domain 으로 본다. Spring Batch COMPLETED 와 업무 대사 성공을 분리한다. Center-Cut 은 FAILED/UNKNOWN 대상만 선택적으로 처리한다.

42. ADM Control Plane Architecture

현재 ADM 은 63 개 Route Registry 를 갖고 각 Route 에 Component/Feature Flag/Menu/Risk/expectedOperationIds 를 연결한다. Router 는 Public status route 를 제외하고 인증된 Session Projection 을 기준으로 접근권한과 Feature 활성화를 재평가한다.

Control Plane 은 다음 계층으로 분리한다.

Vue Component
 -> Generated/typed API or explicit Operation binding
 -> ADM Backend Controller
 -> Owner Query/Command Port
 -> Owner state

expectedOperationIds 는 Route 가 기대하는 Backend contract 목록이며 화면 Button 정의 그 자체가 아니다. 실제 Field/Button/활성조건은 Component Source 를 정본으로 한다.

43. ADM 위험조치 Architecture

위험조치는 Route Risk 와 별도로 Action-level Permission 을 적용한다. 공통 축은:

- authenticated operator.
- button/action grant.
- target snapshot.
- reason.
- expected version/CAS when applicable.
- approval/break-glass when policy requires.
- idempotency/operation id for response loss.
- immutable audit.

Current Component-level static catalog 는 63/63 을 ADM_COMPONENT_CONTRACT_CATALOG.csv 로 추적한다. Browser runtime 검증은 별도 Evidence 다.

44. BZA Architecture

BZA 는 26 Route 의 조직·직원·Assignment·사용자·Role·Permission·Data Scope·Approval·Delegation·Session·Audit·Attachment·Download 를 제공한다. Effective date/as-of 조회를 중요한 시간 축으로 사용한다.

BZA Approval 은 Owner 업무 상태를 직접 수정하기보다 승인 Snapshot 과 실행권한을 제공하며 실제 Owner Command 실패/UNKNOWN 시 Compensation/Reconcile 을 연결한다.

45. Gateway Data Plane Architecture

Listener

- > Route Predicate
- > Rewrite/Header policy
- > Authentication/Authorization
- > Nonce/HMAC/Audience/Body Hash
- > Rate/Bulkhead/Circuit
- > Target selection/Load balance
- > Attempt/Timeout/Retry
- > Response/Audit/Trace

외부 공개는 Default Deny 다. ACTIVE Binding, 승인된 Security Policy, 적용된 Gateway Instance 상태가 있어야 한다.

46. Gateway Control Plane Architecture

Draft Binding/Group

- > Validate
- > Approval
- > Publish version/checksum
- > Instance apply
- > ACK/NACK
- > observedVersion/drift
- > Reconcile or LKG Rollback

Control Plane 장애가 Data Plane 의 Last Known Good 를 바로 제거하지 않게 LKG 를 둔다. Partial Apply 는 Instance 별 expected/applied version 으로 관찰한다.

47. Runtime Control Architecture

Runtime Control 은 local controller 또는 remote HTTP control plane 구성을 지원하며 desired change 와 instance agent 를 분리한다. Agent 등록에는 instance/service/endpoint/environment/zone/cell/artifact/schema/config hash/capability manifest 를 포함할 수 있다.

Apply Guard 는 timeout/max attempts/backoff/jitter/concurrency/circuit 과 target snapshot 을 사용한다. Remote control plane 에는 agent token 이 필요하고 Agent inbox path 는 지정 root 밖으로 벗어날 수 없게 검증한다.

48. Configuration Architecture 상세

Configuration 은 네 층으로 구분한다.

1. **Capability selection**: Starter/Profile Catalog.
2. **Typed runtime key**: ConfigurationProperties/@Value.
3. **Secret reference**: Provider-managed secret.

4. **Dynamic desired/observed**: Runtime Control 변경 상태.

이 네 층을 한 application.yml 목록으로 단순화하지 않는다. 현재 243 Source-confirmed leaf 는 PROPERTY_LEAF_CATALOG.csv 에서 exact Source Path 와 함께 관리한다.

49. DB Vendor Architecture

Runtime DB 는 cpf.db.vendor 와 cpf.db.resource-root 로 중앙 Vendor Pack 을 선택한다. Pack 은 pack.json, migration, runtime mapper/repository resource 를 포함하며 path traversal/symlink escape 를 검사한다.

```
MariaDB Pack -----PostgreSQL Pack -----> CpfVendorResourceRoot -> CpfSqlResourceResolver -> MyBatis/Repository/Flyway
Oracle Pack -----/
```

다른 Vendor resource fallback 을 금지해 잘못된 SQL 이 조용히 실행되는 것을 방지한다.

50. Data Model Architecture

Owner 별로 Current/History/Idempotency/Attempt/Outbox/Inbox/DLQ/Approval Snapshot/Audit/Reconciliation Decision 패턴을 조합한다. 모든 기능에 모든 Table 을 강제하는 것이 아니라 Failure/Recovery 요구가 있는 곳에 해당 원장을 둔다.

51. Security Architecture 상세

Browser

Server Session Handle, Credential Vault, CSRF Token, Trusted Origin, CSP/HSTS/Frame deny 를 사용한다. Session cookie timeout/max-session/TTL 의 Source validation 을 따른다.

Resource API

JWT issuer 또는 JWK Set 중 정확히 하나를 사용하고 Audience Validator 를 적용한다.

Service Identity

Active/previous key 를 통한 rotation grace 와 짧은 TTL 을 사용한다. Secret 은 Raw config 가 아니라 승인 Provider 로 이동시키는 것을 기준으로 한다.

Operator

Menu/Route/API/Button/Scope/Masking, Approval, Break-glass, Audit 를 계층적으로 적용한다.

52. Secret Lifecycle Architecture

Secret Starter 는 승인된 CpfSecretProvider 가 없으면 제품 Runtime 기동을 거부한다. ADM Secret 화면은 raw secret 을 반환하지 않고 Provider Reference/Metadata/Version/Expiry/Rotatable 속성만 본다. Rotation 은 Provider 와 Secret 양쪽이 지원하는 경우에만 요청한다.

53. Observability Architecture 상세

관측 상관관계:

```
Transaction ID
```

- + Trace ID
- + Business Transaction ID
- + Operation ID
- + Attempt ID
- + Batch Execution/Step
- + Gateway/External Tracking
- + Actor/Audit

Log Policy 는 runtime cache 와 versioned managed policy 를 구분하고 Capture Mode/Allowlist/Byte ceiling/Masking/Sampling/Retention 을 적용한다. Metric 은 PII/고카디널리티 Label 남용을 피한다.

54. Approval·Break-glass Architecture

Approval Policy 는 Versioned policy/steps/decision rule/self-approval/break-glass 허용 여부를 가진다. Request 에는 target/payload snapshot 과 request/idempotency key 를 연결한다. Owner Command 실행 후 결과가 UNKNOWN 이면 같은 Mutation 을 재실행하지 않고 Owner state reconciliation 을 수행한다.

Break-glass 는 좁은 Scope 와 짧은 TTL, 종료, Post Review 를 갖는다. 현재 ADM 화면은 TTL 1~30 분 범위를 사용한다.

55. Artifact·Supply-chain Architecture

Source SHA
 → Toolchain/BOM/Lock
 → Build
 → Artifact Hash
 → SBOM/Provenance
 → Registry/Offline Bundle
 → Deployment target
 → Runtime observed artifact version/commit

Artifact 와 DB Migration, Config, Generated Client 를 같은 Version 문자열만으로 묶지 않고 Hash/Manifest 로 연결한다.

56. Deployment Topology

Topology	추가 검토
Modular Monolith	module boundary violation, local call semantics
Microservice	remote identity, timeout, service registry, network failure
External WAS	artifact/config/session separation
Multi-instance	load balancing, idempotency, lease/fencing, config drift
Multi-zone	zone-aware target, partial partition, failover
Batch Worker Farm	queue/lease/fencing/backpressure
Gateway Fleet	apply ACK/NACK, version drift, LKG

57. Rolling·Blue-Green·Canary Architecture

배포 방식은 이름보다 mixed-version 계약으로 검토한다. New/Old Application 이 동시에 DB/Message/API 를 사용할 수 있는지, Config/Feature Flag 가 어떤 순서로 전환되는지, Target 별 observed version 이 어떻게 확인되는지를 정의한다.

Canary 는 작은 Traffic 비율만 의미하지 않고 Error/Latency/Audit/Business reconciliation 을 비교할 지표가 있어야 한다.

58. Capacity·Backpressure Architecture

모든 queue/thread/connection/file/page/export/replay 에 상한을 둔다. Batch/Message/File 의 대용량은 producer 속도가 consumer capacity 를 넘을 때 backpressure 가 동작하도록 한다. Capacity 초과를 무제한 heap/queue 증가로 흡수하지 않는다.

59. Failure Domain Matrix

Failure Domain	감지	격리/보호	복구	종결 Evidence
DB conflict/deadlock	exception/version	rollback/CAS	bounded retry/re-read	owner version/audit
Broker duplicate/loss	inbox/outbox/attempt	idempotency/lease	replay/reconcile	ledger+business state
External response loss	timeout/attempt	UNKNOWN_RESULT	status query/compensation	target evidence
Worker kill	heartbeat/lease	fencing	restart/reassign	metadata+business reconcile
Gateway partial apply	ACK/NACK/drift	LKG	reapply/rollback	observed version
Runtime config partial	target status	version/checksum	reconcile/LKG	desired=observed
DB migration partial	migration/verify	maintenance/backup	forward fix/restore	schema+runtime query
Secret rotation partial	provider metadata/health	old/new key grace	retry/rollback	consumer health/audit

60. Recovery Architecture 원칙

1. Side Effect 이전/이후를 구분한다.
2. 결과 Evidence 가 없으면 실패로 단정하지 않는다.
3. UNKNOWN 상태에서 blind retry 보다 status/reconcile 을 우선한다.
4. 다중 Target 은 Target 별 결과를 보존한다.
5. Compensation 은 원래 Transaction rollback 과 다름을 표시한다.
6. Manual Decision 은 Operator/Reason/Approval/Audit 를 남긴다.

61. DR Architecture

DR 은 Backup 보유가 아니라 Restore/Failover/Failback 과 Application/Broker/DB/Secret/Gateway 의 상태 정합성을 포함한다. RPO/RTO 는 실제 Drill 에서 측정해야 한다. 이 문서 작성 세션에서는 DR Drill 을 수행하지 않았으므로 미검증이다.

62. EDU Reference Architecture

135 EDU 는 제품 기능 수를 과장하는 카탈로그가 아니라 정상·실패·복구 계약의 실행 Reference 다.

Category	수량	주 Architecture 축
DEV	45	Domain/API/transaction/integration/security/db
BAT	30	Spring Batch/scheduler/worker/center-cut/recovery

Category	수량	주 Architecture 축
ADM	17	owner port/permission/version/unknown/frontend
BZA	14	organization/permission/approval/delegation
GW	14	route/security/publish/LKG/reconcile
OPS	15	install/config/db/broker/deploy/backup/DR

Manual/Catalog/Handler/Consumer/Test 의 필드 동등성은 별도 QA 가 추적한다.

63. Tooling Architecture

Customer Entry Tool	역할	Manual
cpf-tools/scripts/sync-database-artifacts.ps1	공식 3-Vendor Source/Pack/Schema/Query/Checksum 을 정본에서 재생성·대조	01:05
cpf-tools/scripts/initialize-cpf-database.ps1	Platform Pack 을 Vendor 별 신규 설치·검증하고 선택 Module 을 실행	01:05
cpf-tools/generator/create-domain.ps1	Domain/Profile/Capability/Provider/DB 를 검증하고 생성	01:05
cpf-tools/scripts/check-work-context.ps1	정본 문서, HEAD/Branch/Working Tree, Current Request 기준선 확인	01:05
cpf-tools/scripts/sync-generated-domain-artifacts.ps1	ownership hash 와 재생성 결과를 비교해 drift 를 검출/선택 반영	01:05
cpf-tools/scripts/check-generator-arbitrary-domain-parity.ps1	서로 다른 두 Domain 을 격리 생성해 normalized parity 검증	01:05
cpf-tools/scripts/verify-full-product.ps1	Static/Gradle/Frontend/DB/Generator/Browser/Evidence Gate 통합	01:05
cpf-tools/build/gradle-plugin	Java/Dependency/Build/Publication 공통 규칙	01:05
cpf-tools/build/platform-bom	CPF 와 외부 Dependency 버전 정렬	01:05

Tool 은 Product Runtime 과 별도 Control Surface 다. Tool 이 생성한 Source/DB/Artifact 를 실제 Runtime 이 Consumer 하는지 검증해야 하며 Tool 자체 성공 로그만으로 제품 동작을 판정하지 않는다.

64. Architecture Source Trace

Surface	Source	Documents	Basis
Top target	cpf-docs/governance/CPF_FINAL_TARGET_REQUIREMENTS.md	00:01:02:03:04:05:90:91:design	4870b2073387...
Documentation standard	cpf-docs/specification/CPF_DOCUMENTATION_STANDARD.md	all official docs	4870b2073387...
Starter catalog	cpf-tools/generator/contracts/cpf-starter-catalog.json	00:01:05	4870b2073387...
Generator	cpf-tools/generator/create-domain.ps1	01	4870b2073387...
Tool registry	cpf-tools/README.md	01:05	4870b2073387...
Full verification	cpf-tools/scripts/verify-full-product.ps1	05	4870b2073387...

Surface	Source	Documents	Basis
DB sync	cpf-tools/scripts/sync-database-artifacts.ps1	05:DB-standard	4870b2073387...
DB initialize	cpf-tools/scripts/initialize-cpf-database.ps1	05:DB-standard	4870b2073387...
Generated Domain sync	cpf-tools/scripts/sync-generated-domain-artifacts.ps1	01:05	4870b2073387...
Arbitrary Domain parity	cpf-tools/scripts/check-generator-arbitrary-domain-parity.ps1	01:05	4870b2073387...
EDU catalog	cpf-reference/src/main/resources/edu/manual-135-catalog.json	01:02:03:05:90:91	4870b2073387...
EDU process runner	cpf-reference/src/main/scripts/edu/invoke-reference-edu.ps1	01:05	4870b2073387...
ADM logs UI	cpf-admin/frontend/src/features/logs/LogsPage.vue	04	4870b2073387...
BZA organizations UI	cpf-biz-admin/frontend/src/features/organizations/OrganizationsPage.vue	90	4870b2073387...
Stack	gradle/cpf-stack.properties	00:01:05:design	4870b2073387...
Build graph	settings.gradle	00:01:02:05:design	4870b2073387...

65. Architecture 변경 영향 절차

Architecture 변경은 다음 순서로 검토한다.

1. 변경할 Requirement 와 현재 Owner 를 확인한다.
2. Consumer 와 Public API/SPI/Internal 영향을 열거한다.
3. Data/Transaction/Message/Config/Frontend/Batch/Gateway 흐름을 비교한다.
4. 정상 흐름과 최소 1 개 현실적인 Failure Path 를 그린다.
5. Security/Permission/Secret/Approval/Audit 영향을 검토한다.
6. Local/Remote/Multi-instance/DB3/Mixed Version 영향을 검토한다.
7. Recovery/Compensation/LKG/Restore 경로를 확정한다.
8. Source/SQL/API/Test/Guide/Design/Evidence 를 함께 변경한다.
9. 실행하지 못한 Runtime/Browser/DB/Fault 검증을 미검증으로 기록한다.

66. 현재 Architecture 검증 상태

영역	이 문서 세션 검증
Source/Route/Config/EDU/Starter 정적 대조	완료
ADM 63 Component-level 계약 대조	완료 (source static)
BZA 26 Route 계약 대조	완료 (source static)
Runtime leaf 243 Source path 대조	완료 (source static)
Java/Gradle 전체 Build	미검증

영역	이 문서 세션 검증
ADM/BZA Browser 실제 Session E2E	미검증
MariaDB/PostgreSQL/Oracle Live lifecycle	미검증
Broker/Multi-process/Lease fault	미검증
Rolling/Blue-Green/Canary 실제 배포	미검증
Backup/Restore/DR Drill	미검증

Architecture 문서는 구현 목표와 현재 Evidence 범위를 분리해 읽어야 한다.

67. Product/EDU Ownership - 07_06 현행 기준

ADM·BZA·Gateway·Batch Runtime 은 제품 Module 이 소유하고, EDU 는 Public API·SPI·Extension·Integration Consumer 를 교육한다. Product 내부 기능을 이름만 바꾼 Generic Handler 로 Reference 에 복제하지 않는다. 기존 EDU ID 의 유지/통합/Product 귀속은 R6J Architecture QA 결과로 결정한다.

68. End-to-End Transaction Lineage Architecture

최상위 transactionId 를 Data Plane 전체에서 유지하고 segmentId/parentSegmentId/attempt/tracelId/spanId 로 호출 계층과 재시도를 표현한다. Batch/외부 연계는 전용 실행·원격 식별자를 원 transactionId 에 연결한다. File Log 와 DB Timeline 은 같은 거래를 가리키되 민감 Payload 를 중복 보관하지 않는다. ADM 은 transactionId 하나로 이 계보를 조합하되 Source 누락 시 Partial/Stale 을 노출한다. 종단간 Runtime 검증은 미검증이다.

69. 현재 제품 검증 상태와 R6J 사용 경계

07_06 Final QA 중앙 Merge: 현재 master 는 07_05 제품 Source 위에 QA/Control 산출물을 추가한 Commit 이며 Product Source 변경은 없다. Final verdict 는 FAIL / REDEVELOPMENT REQUIRED + UNVERIFIED / RELEASE_BLOCKED 다. Canonical 완료 분모는 169 Requirement 이고, 신규 독립 개발 Action 은 31 개(P0 22 / P1 9)다. 전체 Scope 는 169 Requirement + 기존 56 Finding + 신규 31 + self-found + Runtime 13 + 개발 중 추가 발견 전체이며, 34/34 direct rework 구현 사실만으로 완료 판정하지 않는다.

Architecture Final QA 결정: cpf-core 는 persistence 구현을 소유하지 않고, cpf_transaction_lineage 는 normalized operational projection/index 이며 dual-primary 가 아니다. PRODUCT_ADM/MERGE_EDU 13 개는 runtime handler 가 아닌 reference/redirect metadata 로 비실행화해야 한다.

- 현재 Architecture 정본은 제품/EDU 경계와 transactionId/File/DB logging 요구를 포함한다.
- 07_05 Source 에 EDU-ADM 17 의 9/4/4 Architecture Decision 이 반영됐다: PRODUCT_ADM 9, EXTENSION_SAMPLE 4, MERGE_EDU 4. Product/Merge 항목은 executable=false 로 정리하고 Extension Sample 만 실제 Public Extension consumer 교육으로 유지한다.
- 07_05 Source 는 CpfTransactionTimelineQueryFacade 의 one-shot lineage 와 DB3 transaction_lineage 를 추가해 LOCAL/REMOTE/MESSAGE/DLQ/FILE/UNKNOWN/TRACE/AUDIT 등의 연결을 보강했다. 2+ instance·Broker·Batch·ADM 을 포함한 실제 end-to-end lineage 는 Runtime 미검증이다.
- 현재 master 4870b2073387(07_06)는 07_05 Product Source rework 와 07_06 Final QA/Control 판정을 함께 포함한다. Architecture 9/4/4 EDU 재분류, transaction lineage, FileLog recovery 와 action permission 변경을 이 문서에 반영한다.

마지막 확인 - 이 문서를 닫기 전에

- ✓ Module Ownership 과 의존 방향에 역방향/순환이 없다.
- ✓ Local/Remote·MSA/Modular Monolith 계약 동등성을 유지한다.
- ✓ transactionId·UNKNOWN·Recovery·Audit 흐름이 Owner 경계를 넘을 때 추적된다.
- ✓ Runtime/DR/DB3/Browser 검증이 끝나기 전 제품 완료로 확대하지 않는다.